Proof Verification Summary

Mathematical Statement:

a+b=b+a

Total Iterations:

20

Result:

A valid proof has been found!

Latest Iteration

Latest Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b =
cong succ (add-comm a b) `trans` sym (add-succ b a)
```

Latest Verification Result:

Proof is valid.

Feedback:
Proof has been compiled and verified after 20 iterations.

Iteration 1

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = cong (add zero) refl
add-comm (succ a) b = cong succ (add-comm a b)

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl
```

Verification Result:

Proof is invalid.

```
Feedback:
Not in scope error: Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:15,19-23
Not in scope:
cong at C:\Users\Daniel\Desktop\temp_proof.agda:15,19-23
when scope checking cong
```

Iteration 2

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = refl
add-comm (succ a) b = cong succ (add-comm a b)
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:18,19-23
b != add b zero of type ℕ
when checking that the expression refl has type
add zero b ≡ add b zero

Iteration 3

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = add-zero b
add-comm (succ a) b = cong succ (add-comm a b)
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:26,19-29
add b zero != b of type ℕ
when checking that the expression add-zero b has type
add zero b ≡ add b zero

Iteration 4

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
```

```
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = add-zero b
add-comm (succ a) b = cong succ (add-comm a b) `trans` add-succ b a
where
_`trans`_ : {x y z : ℕ} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q
```

Verification Result:

Proof is invalid.

Feedback:
```
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:26,19-29
add b zero != b of type ℕ
when checking that the expression add-zero b has type
add zero b ≡ add b zero
```

Iteration 5

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = cong succ (add-comm a b) `trans` add-succ b a
```

```
sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {x y z : ℕ} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q
```

Verification Result:

Proof is invalid.

Feedback:
Not in scope error: Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:26,19-22
Not in scope:
sym at C:\Users\Daniel\Desktop\temp_proof.agda:26,19-22
when scope checking sym

Iteration 6

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = cong succ (add-comm a b) `trans` add-succ b a
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:33,56-68
add b (succ a) != succ (add b a) of type ℕ
when checking that the expression add-succ b a has type

succ (add b a) ≡ add b (succ a)

Iteration 7

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = cong succ (add-comm a b) `trans` sym (add-succ b a)
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
Verification failed: The statement was not confirmed as valid by the API.

Iteration 8

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)
```

```
data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

comm-helper : (a b : ℕ) → add (succ a) b ≡ succ (add b a)
comm-helper a zero = refl
comm-helper a (succ b) = cong succ (comm-helper a b) `trans` sym (add-succ b a)

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = comm-helper a b `trans` sym (add-succ b a)
```

Verification Result:

Proof is invalid.

```
Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:32,22-26
add a zero != a of type ℕ
when checking that the expression refl has type
add (succ a) zero ≡ succ (add zero a)
```

Iteration 9

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
```

```
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

comm-helper : (a b : ℕ) → add (succ a) b ≡ succ (add b a)
comm-helper a zero = sym (add-zero (succ a))
comm-helper a (succ b) = cong succ (comm-helper a b) `trans` sym (add-succ b a)

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = comm-helper a b `trans` sym (add-succ b a)
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:32,27-44
add a zero != a of type ℕ
when checking that the expression add-zero (succ a) has type
succ (add zero a) ≡ add (succ a) zero

Iteration 10

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

comm-helper : (a b : ℕ) → succ (add a b) ≡ add b (succ a)
comm-helper zero b = sym (add-succ b zero)
comm-helper (succ a) b = cong succ (comm-helper a b) `trans` sym (add-succ b (succ a))

add-comm : (a b : ℕ) → add a b ≡ add b a
```

```
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = comm-helper a b `trans` sym (add-succ b a)
```

Verification Result:

Proof is invalid.

Feedback:
```
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:32,27-42
add b zero != b of type ℕ
when checking that the expression add-succ b zero has type
add b (succ zero) ≡ succ (add zero b)
```

Iteration 11

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

comm-helper : (a b : ℕ) → succ (add a b) ≡ add b (succ a)
comm-helper zero b = add-zero (succ b) `trans` sym (add-succ zero b)
comm-helper (succ a) b = cong succ (comm-helper a b) `trans` sym (add-succ b (succ a))

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = comm-helper a b `trans` sym (add-succ b a)
```

Verification Result:

Proof is invalid.

Feedback:
```
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:32,22-39
add b zero != b of type ℕ
when checking that the expression add-zero (succ b) has type
```

succ (add zero b) ≡ _y_50

Iteration 12

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

comm-helper : (a b : ℕ) → succ (add a b) ≡ add (succ b) a
comm-helper zero b = add-succ b zero `trans` cong succ (sym (add-zero b))
comm-helper (succ a) b = cong succ (comm-helper a b) `trans` sym (add-succ b (succ a))

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = comm-helper a b `trans` sym (add-succ b a)
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:32,22-37
add b (succ zero) != succ (add zero b) of type ℕ
when checking that the expression add-succ b zero has type
succ (add zero b) ≡ _y_50

Iteration 13

Proof:

```
module temp_proof where
```

```
data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

comm-helper : (a b : ℕ) → succ (add a b) ≡ add (succ b) a
comm-helper zero b = sym (add-succ b zero)
comm-helper (succ a) b = cong succ (comm-helper a b) `trans` sym (add-succ b (succ a))

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = comm-helper a b `trans` sym (add-succ b a)
```

Verification Result:

Proof is invalid.

```
Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:32,27-42
add b (succ zero) != succ (add b zero) of type ℕ
when checking that the expression add-succ b zero has type
add (succ b) zero ≡ succ (add zero b)
```

Iteration 14

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl
```

```
add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

comm-helper : (a b : ℕ) → add (succ a) b ≡ succ (add a b)
comm-helper a zero = refl
comm-helper a (succ b) = cong succ (comm-helper a b)

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = comm-helper a b `trans` sym (add-succ b a)
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:33,37-52
succ (add a b) != add a (succ b) of type ℕ
when checking that the expression comm-helper a b has type
add a (succ b) ≡ add a (succ b)

Iteration 15

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
```

```
refl `trans` q = q

comm-helper : (a b : ℕ) → add (succ a) b ≡ add a (succ b)
comm-helper a zero = refl
comm-helper a (succ b) = cong succ (comm-helper a b)

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = comm-helper a b `trans` cong succ (add-comm a b)
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:32,22-26
succ (add a zero) != add a (succ zero) of type ℕ
when checking that the expression refl has type
add (succ a) zero ≡ add a (succ zero)

Iteration 16

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

comm-helper : (a b : ℕ) → add (succ a) b ≡ add a (succ b)
comm-helper zero b = add-succ zero b
comm-helper (succ a) b = cong succ (comm-helper a b)

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = comm-helper a b `trans` cong succ (add-comm a b)
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:37,47-70
succ _x_64 != add a (succ b) of type ℕ
when checking that the inferred type of an application
succ _x_64 ≡ succ _y_65
matches the expected type
add a (succ b) ≡ add b (succ a)

Iteration 17

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

comm-helper : (a b : ℕ) → succ (add a b) ≡ add a (succ b)
comm-helper zero b = sym (add-succ zero b)
comm-helper (succ a) b = cong succ (comm-helper a b)

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = comm-helper a b `trans` cong (λ x → succ x) (add-comm a b)
```

Verification Result:

Proof is invalid.

Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:37,47-80
succ _x_67 != add a (succ b) of type ℕ
when checking that the inferred type of an application
succ _x_67 ≡ succ _y_68
matches the expected type
add a (succ b) ≡ add b (succ a)

Iteration 18

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

comm-helper : (a b : ℕ) → add (succ a) b ≡ succ (add a b)
comm-helper zero b = refl
comm-helper (succ a) b = cong succ (comm-helper a b)

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = comm-helper a b `trans` cong (λ x → succ x) (add-comm a b)
```

Verification Result:

Proof is invalid.

```
Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:37,47-80
succ _y_65 != add b (succ a) of type ℕ
when checking that the inferred type of an application
succ (add a b) ≡ succ _y_65
matches the expected type
succ (add a b) ≡ add b (succ a)
```

Iteration 19

Proof:

```
module temp_proof where
```

```
data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)

sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b = cong succ (add-comm a b) `trans` sym (add-succ b a)
```

Verification Result:

Proof is invalid.

Feedback:
Failed to refine proof, refining proof and retrying..

Iteration 20

Proof:

```
module temp_proof where

data ℕ : Set where
zero : ℕ
succ : ℕ → ℕ

add : ℕ → ℕ → ℕ
add zero n = n
add (succ m) n = succ (add m n)

data _≡_ {A : Set} (x : A) : A → Set where
refl : x ≡ x

cong : {A B : Set} {x y : A} → (f : A → B) → x ≡ y → f x ≡ f y
cong f refl = refl

add-zero : (a : ℕ) → add a zero ≡ a
add-zero zero = refl
add-zero (succ a) = cong succ (add-zero a)

add-succ : (a b : ℕ) → add a (succ b) ≡ succ (add a b)
add-succ zero b = refl
add-succ (succ a) b = cong succ (add-succ a b)
```

```
sym : {A : Set} {x y : A} → x ≡ y → y ≡ x
sym refl = refl

_`trans`_ : {A : Set} {x y z : A} → x ≡ y → y ≡ z → x ≡ z
refl `trans` q = q

add-comm : (a b : ℕ) → add a b ≡ add b a
add-comm zero b = sym (add-zero b)
add-comm (succ a) b =
cong succ (add-comm a b) `trans` add-succ b a
```

Verification Result:

Proof is invalid.

```
Feedback:
Checking temp_proof (C:\Users\Daniel\Desktop\temp_proof.agda).
C:\Users\Daniel\Desktop\temp_proof.agda:34,36-48
add b (succ a) != succ (add b a) of type ℕ
when checking that the expression add-succ b a has type
succ (add b a) ≡ add b (succ a)
```
